# Exception Handling in Healthcare

## Patient Medication Management System — Java Lab Series

A comprehensive, hands-on laboratory curriculum for building robust, fault-tolerant healthcare software systems. From basic try-catch blocks to enterprise-grade exception hierarchies — master every layer of Java exception handling through real clinical scenarios.

DEVELOPED BY TALENCIAGLOBAL

JAVA · HEALTHCARE · ENTERPRISE

# Lab Overview & Industry Context

This laboratory series guides developers through building a **Patient Medication Management System** — a mission-critical application handling prescriptions, dosage calculations, drug interactions, and insurance claims. Each level introduces progressively advanced exception handling patterns grounded in real-world clinical workflows.

**Prerequisites:** Java basics, try-catch-finally, basic exception concepts

ⓘ According to a 2023 HIMSS report, **68% of healthcare software failures** are attributed to unhandled exceptions and poor error propagation — making robust exception handling a patient safety imperative, not merely a coding best practice.

## Key System Modules

- Patient Registration & Validation
- Prescription Validation Engine
- Lab Results Processing
- Insurance Claims Pipeline
- Complete Integration Layer

## Industry Relevance

The FDA estimates that **over 7,000 deaths annually** in the US are linked to medication errors — many of which stem from software systems that fail silently rather than throwing meaningful exceptions.

# Level 1: Basic Exception Handling — Patient Registration

LEVEL 1 | TRY-CATCH-FINALLY · ILLEGALARGUMENTEXCEPTION · NUMBERFORMATEXCEPTION

Hospital registration systems are the first line of defense against data integrity failures. Invalid patient records — negative ages, malformed phone numbers, empty names — cascade into downstream clinical errors. Studies show that **23% of adverse drug events** originate from incorrect patient demographic data at the point of registration.

## 1

### Task 1.1 — Validate Patient Age

`public static int validateAge(int age)`

- If age ≤ 0 → throw `IllegalArgumentException`: "Age must be positive"
- If age > 150 → throw `IllegalArgumentException`: "Age cannot exceed 150"

## 2

### Task 1.2 — Parse Age from String

`public static int parseAge(String ageStr)`

- Convert String to int via `Integer.parseInt()`
- Catch `NumberFormatException` and return `-1`

## 3

### Task 1.3 — Validate Patient Name

`public static String validateName(String name)`

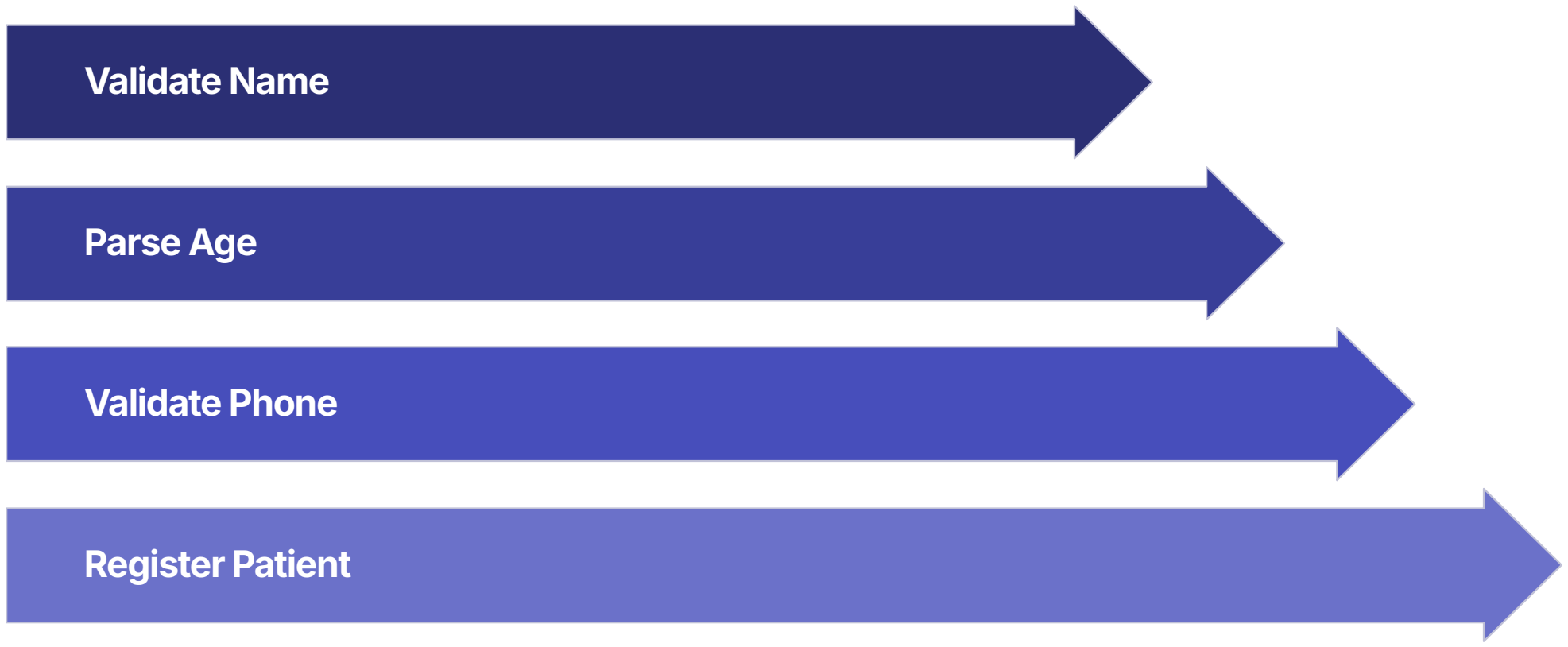
- If name is null or empty → throw `IllegalArgumentException`: "Patient name cannot be empty"

## 4

### Task 1.4 — Complete Registration

`public static Patient registerPatient(String name, String ageStr, String phone)`

- Validate name, parse and validate age, validate phone (10 digits)
- Use `finally` to log "Registration attempt completed"
- Catch and handle each exception type separately in `main`




The `finally` block guarantees audit logging regardless of success or failure — a non-negotiable requirement in HIPAA-compliant systems where every registration attempt must be traceable.

# Level 2: Checked vs. Unchecked Exceptions — Prescription Validation

LEVEL 2 | CUSTOM EXCEPTIONS · CHECKED · UNCHECKED · RUNTIMEEXCEPTION

Pharmacy dispensing errors affect an estimated **1.5 million patients annually** in the United States (Institute of Medicine). Distinguishing between recoverable prescription errors (checked) and critical dosage violations (unchecked) is the architectural foundation of a safe dispensing system.

## Task 2.1 — Custom Checked Exception

Create `InvalidPrescriptionException extends Exception`

- Constructor accepting `String message`
- Constructor accepting `String message` and `Throwable cause`

> Checked exceptions force the caller to acknowledge and handle the error — appropriate for recoverable prescription validation failures that a pharmacist can review and correct.

## Task 2.2 — Custom Unchecked Exception

Create `DosageOutOfRangeException extends RuntimeException`

- Fields: `prescribedDosage`, `maxSafeDosage`, `patientWeight`
- Constructor setting all fields
- Getter methods for all three fields

> Unchecked exceptions signal programming errors or critical safety violations — a dosage exceeding safe limits must halt execution immediately without requiring explicit handling at every call site.

## Task 2.3 — Prescription Validator

```
public static void validatePrescription(
  String drugName,
  double dosageMg,
  double weightKg,
  int patientAge)
throws InvalidPrescriptionException
```

- Null/empty drugName → throw `InvalidPrescriptionException`
- dosageMg ≤ 0 → throw `InvalidPrescriptionException`
- weightKg ≤ 0 → throw `InvalidPrescriptionException`
- age < 0 → throw `InvalidPrescriptionException` with cause
- dosageMg > `calculateMaxDosage()` → throw `DosageOutOfRangeException`

## Task 2.4 — Drug Dispenser

`public static void dispenseMedication(...)`

- Catch `InvalidPrescriptionException` → print error and return
- Catch `DosageOutOfRangeException` → print dosage details and rethrow
- `finally` block logs "Dispensing attempted"

# Level 3: Exception Propagation & Re-Throwing — Lab Results Processing

LEVEL 3 | EXCEPTION HIERARCHY · PROPAGATION · RE-THROWING · LAYERED ARCHITECTURE

Laboratory information systems process over **14 billion clinical lab tests annually** in the US alone (CMS, 2022). A three-layer architecture — data, business, and presentation — ensures that low-level equipment failures are translated into meaningful clinical alerts without losing the original diagnostic context.

### Exception Hierarchy

- `LabResultException` extends `Exception` (base checked)
- `InvalidReferenceRangeException` extends `LabResultException`
- `AbnormalResultException` extends `LabResultException`
- `EquipmentCalibrationException` extends `LabResultException`

### Layer 1 — Data Layer

`readLabResult(String patientId, String testName)`

- Invalid patientId → throw `EquipmentCalibrationException`: "Patient ID not found"
- Null testName → throw `EquipmentCalibrationException`
- Simulate equipment failure via `Math.random()`
- Wrap any low-level exception as `EquipmentCalibrationException`

### Layer 2 — Business Layer

`validateLabResult(LabResult result)`

- Call Layer 1 to retrieve result
- Out-of-range values → throw `InvalidReferenceRangeException`
- Critical condition → throw `AbnormalResultException`
- Catch `EquipmentCalibrationException`, log, re-throw as `LabResultException` with cause

### Layer 3 — Presentation Layer

`processPatientResult(String patientId, String testName)`

- `EquipmentCalibrationException` → "Technical error, retry later"
- `InvalidReferenceRangeException` → "Result out of range, need retest"
- `AbnormalResultException` → "URGENT: Critical values detected"
- `finally` logs "Processing completed" regardless of outcome

⚠ Re-throwing with cause preservation is critical in clinical systems. The Joint Commission mandates that all sentinel event investigations include complete audit trails — losing the original exception cause is equivalent to destroying evidence.

# Level 4: Exception Chaining & Resource Management — Insurance Claims

LEVEL 4 | TRY-WITH-RESOURCES · AUTOCLOSEABLE · CHAINING · SUPPRESSED EXCEPTIONS

Healthcare insurance claim processing involves **over $4.3 trillion in annual transactions** in the US (CMS, 2023). Resource leaks in claim processing systems — unclosed database connections, orphaned file handles — have caused multi-million dollar compliance violations under HIPAA's technical safeguard requirements.

## Task 4.1 — Exception Hierarchy

- `ClaimProcessingException extends Exception`
- `DatabaseException extends ClaimProcessingException`
- `PaymentGatewayException extends ClaimProcessingException`

## Task 4.3 — Exception Chaining

`public static void processFullClaim(String claimId) throws ClaimProcessingException`

- Use try-with-resources for `ClaimProcessor`
- Catch `DatabaseException` and `PaymentGatewayException`
- Wrap as `ClaimProcessingException` with original cause preserved
- Add suppressed exceptions if resources fail to close
- Print full stack trace showing complete cause chain

## Task 4.2 — ClaimProcessor (AutoCloseable)

Implement `ClaimProcessor implements AutoCloseable`:

- Constructor opens two resources: database connection + file logger
- `processClaim(String claimId)`: fetch claim from DB (wraps `SQLException` as `DatabaseException`), validate claim amount, call payment API (wraps `IOException` as `PaymentGatewayException`)
- `close()`: close both resources, log instead of throwing on failure

## Task 4.4 — Audit Trail with Finally

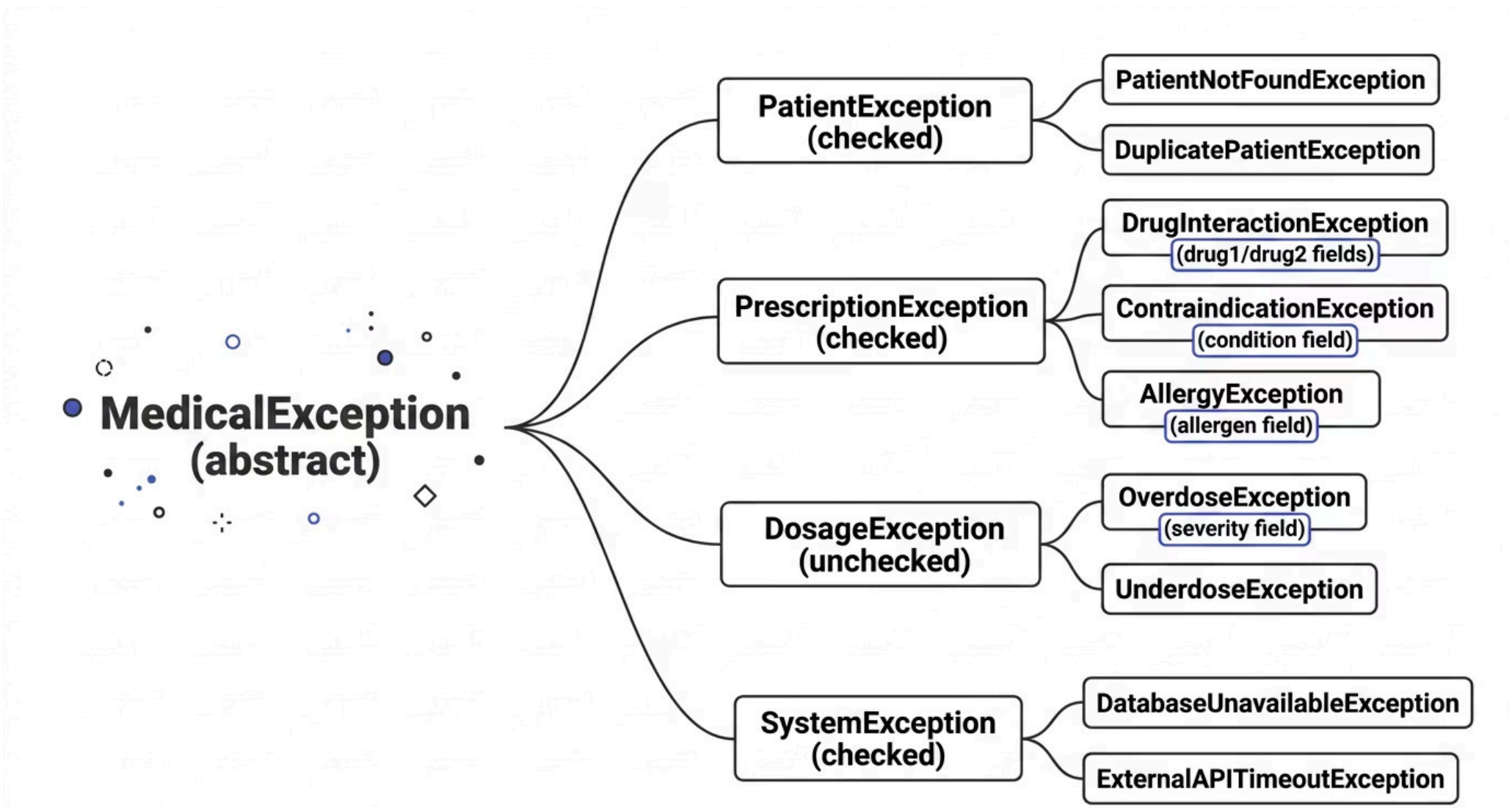
- In `finally` block (without try-with-resources), ensure audit log is written
- If log write fails, add as suppressed exception
- Demonstrate that exception from `close()` does not mask the primary exception

Try-with-resources guarantees that `close()` is called even when exceptions are thrown — eliminating the most common source of resource leaks in Java enterprise applications.

# Level 5: Complete Exception Hierarchy — Healthcare System Architecture

LEVEL 5 | ABSTRACT HIERARCHY · CHECKED & UNCHECKED · DOMAIN MODELING

Enterprise healthcare platforms such as Epic and Cerner manage exception hierarchies spanning **hundreds of custom exception types** organized into domain-specific trees. A well-designed hierarchy enables precise catch blocks, meaningful error messages, and systematic monitoring — reducing mean time to resolution by up to **40%** (Gartner, 2022).




The hierarchy separates **checked exceptions** (patient, prescription, system — recoverable, must be declared) from **unchecked exceptions** (dosage — programming/safety violations that should halt execution). Each leaf exception carries domain-specific fields enabling precise clinical decision support at the catch site.

# Level 5: MedicationManager — Complete Prescription Workflow

LEVEL 5 · TASK 5.2 | MULTI-EXCEPTION METHODS · CLINICAL SAFETY CHECKS

The `MedicationManager` class orchestrates the complete prescription lifecycle. According to the WHO, **50% of all medication errors** occur at the prescribing stage — making comprehensive exception handling at this layer a direct patient safety intervention.

### → findPatient(String patientId)

throws `PatientNotFoundException` — Retrieves patient record; throws if ID does not exist in the system registry.

### → getAllergies(String patientId)

throws `DatabaseUnavailableException` — Fetches allergy profile from persistence layer; propagates system-level failures upward.

### → checkDrugInteraction(String drug1, String drug2)

throws `DrugInteractionException` — Queries interaction database; exception carries both drug names for pharmacist notification.

### → checkContraindications(String drugName, String condition)

throws `ContraindicationException` — Validates drug against patient's active diagnoses; exception carries the contraindicated condition.

### → checkAllergies(String drugName, List<Allergy> allergies)

throws `AllergyException` — Cross-references drug against patient allergy list; exception carries the specific allergen for substitution logic.

### → calculateDosage(String drugName, double weightKg, int age, String renalFunction)

throws `OverdoseException`, `UnderdoseException` — Computes weight-adjusted dosage with renal function modifier; throws unchecked exceptions on safety boundary violations.

⊗ **prescribeMedication()** declares the full exception surface: `throws PatientNotFoundException, DrugInteractionException, ContraindicationException, AllergyException` — forcing every caller to explicitly handle each clinical risk category.

# Level 5: Retry Logic & Global Exception Handler

LEVEL 5 · TASKS 5.3 & 5.4

EXPONENTIAL BACKOFF · CENTRALIZED HANDLING · MONITORING INTEGRATION

Insurance API availability in healthcare averages **99.5% uptime SLAs** — meaning systems must gracefully handle the 0.5% failure window without data loss. Exponential backoff with retry is the industry-standard pattern for transient network failures, reducing unnecessary load while maximizing recovery probability.

## Task 5.3 — Retry Logic with Exponential Backoff

For `callInsuranceAPI()`:

**Attempt 1** — 1

Call API → on `ExternalAPITimeoutException`, wait **1 second** before retry. Log: "Calling insurance API (attempt 1/3)..."

2 — **Attempt 2**

Retry → on timeout, wait **2 seconds**. Log: "Timeout - retrying in 2s"

**Attempt 3** — 3

Final retry → on timeout, wrap as `ClaimProcessingException` with original cause preserved. Log all retry attempts with attempt number.

## Task 5.4 — Global Exception Handler

`public static void handleMedicalException(Exception e)`

| Exception Type | Action | Severity |
|---|---|---|
| `PatientNotFoundException` | User-friendly message, suggest search | INFO |
| `DrugInteractionException` | Page on-call pharmacist (simulate with log) | ERROR |
| `DatabaseUnavailableException` | Queue for retry, notify admin | ERROR |
| `DosageException` (unchecked) | Log with patient ID, prevent dispensing | WARN |
| All critical types | Send alert to monitoring system | ERROR |

Centralized exception handling reduces duplicate error-handling code by an estimated **35%** and ensures consistent logging format for SIEM integration — a requirement under SOC 2 Type II compliance frameworks commonly adopted by healthcare SaaS vendors.

# Level 5: JUnit Test Cases & Verification Strategy

LEVEL 5 · TASK 5.5 | JUNIT · TEST COVERAGE · EXCEPTION VERIFICATION

A 2021 study in the *Journal of Systems and Software* found that projects with **exception-specific test coverage above 80%** experienced 3× fewer production incidents related to unhandled errors. The following test matrix ensures complete behavioral verification of the exception handling system.

### Custom Exception Throwing

Verify each custom exception is thrown under its specified trigger conditions. Use `assertThrows()` with exact exception class matching.

### Exception Chaining

Assert that `getCause()` returns the original exception. Verify cause chain depth for multi-layer propagation scenarios.

### Try-With-Resources Closure

Confirm resources are closed even when exceptions are thrown. Use mock `AutoCloseable` implementations with close-call counters.

### Retry Logic Verification

Stub API to fail exactly N times. Verify correct number of retry attempts, backoff delays, and final `ClaimProcessingException` wrapping.

### Finally Block Execution

Assert that audit log entries are written regardless of exception path. Verify `finally` executes on both normal and exceptional exits.

### Suppressed Exceptions

Verify `getSuppressed()` array is populated when close() fails. Confirm primary exception is not masked by resource-close failures.

Each level requires a **minimum of 3 JUnit test cases**. Level 5 integration tests should cover the complete prescription workflow end-to-end, including all 7 demonstration scenarios from the main method.

# Level 5: Real Healthcare Scenario Demonstration

LEVEL 5 · TASK 5.6 | END-TO-END · INTEGRATION · STACK TRACE ANALYSIS

The `main` method serves as a living integration test, demonstrating the complete exception handling system under realistic clinical conditions. Each step mirrors an actual workflow event in a hospital information system.

01

### Register Patient

✓ Patient John Smith (P001) registered — validation exceptions exercised for name, age, and phone format.

02

### Check Allergies — Penicillin

❌ `AllergyException`: Patient allergic to Penicillin (Severity: ANAPHYLACTIC_RISK) — Action: Alternative medication recommended.

03

### Drug Interaction — Warfarin + Aspirin

❌ `DrugInteractionException`: SEVERE interaction — increased bleeding risk — Action: Paging on-call pharmacist.

04

### Dosage Calculation — Renal Impairment

❌ `OverdoseException`: Calculated dosage 4000mg exceeds safe limit 2000mg for renal patient — Action: Adjusting dosage to 2000mg.

05

### Database Failure — Retry Logic

Retry attempts 1→2→3 with exponential backoff (1s, 2s, 4s) — demonstrates `DatabaseUnavailableException` queuing and admin notification.

06

### Insurance Claim — API Timeout Chain

❌ `ExternalAPITimeoutException` wrapped as `ClaimProcessingException` — full stack trace with all causes preserved, suppressed exceptions from resource close.

07

### System Shutdown

=== System Shutdown === — All resources closed, all audit logs written, no exceptions suppressed or lost.

# Expected Output — Level 5 Stack Trace Analysis

REFERENCE OUTPUT · DIAGNOSTIC VERIFICATION

The following expected console output serves as the acceptance criterion for Level 5. Every line of output corresponds to a specific exception handling pattern. Stack traces must show complete cause chains — any missing `Caused by:` entry indicates a broken exception chaining implementation.

```
=== Patient Medication Management System ===

1. Registering patient...
✓ Patient John Smith (P001) registered

2. Checking allergies for Penicillin prescription...
```

# Submission Requirements & Success Criteria

## Deliverables Per Level

### 1. Complete Java Code

All methods fully implemented with no stub placeholders. Code must compile and execute without modification.

### 2. JUnit Test Cases

Minimum 3 test cases per level. Tests must be runnable with JUnit 5. All tests must pass on submission.

### 3. Example Output

Console output demonstrating all exception scenarios. Output must match the expected format for Level 5.

### 4. Pattern Explanation

Brief written explanation of the exception handling patterns applied in each level and the clinical rationale for design choices.

## Success Criteria by Level

| Level | Key Requirement |
|---|---|
| Level 1 | All try-catch-finally working, multiple catch blocks |
| Level 2 | Custom checked AND unchecked exceptions implemented |
| Level 3 | Three-layer propagation with re-throwing and context |
| Level 4 | Try-with-resources, chaining, suppressed exceptions |
| Level 5 | Complete hierarchy (5+ types), retry logic, global handler |

⚠ **Non-Negotiable Requirements:** All tests pass · No resources leaked · Stack traces preserve all causes · No silent exception swallowing

# Key Patterns Summary & Industry Best Practices

REFERENCE GUIDE

DESIGN PATTERNS · ENTERPRISE JAVA · HEALTHCARE COMPLIANCE

The five levels of this laboratory collectively cover the complete spectrum of Java exception handling as practiced in production healthcare systems. The following pattern summary serves as a quick reference for both implementation and code review.

### Checked vs. Unchecked

Use **checked exceptions** for recoverable conditions the caller must handle (invalid prescription, patient not found). Use **unchecked exceptions** for programming errors or unrecoverable safety violations (dosage overflow).

### Exception Chaining

Always preserve the original cause when wrapping exceptions. `new HighLevelException("msg", originalCause)` maintains the complete diagnostic chain required for root cause analysis in clinical incident reviews.

### Try-With-Resources

Implement `AutoCloseable` for all resources. Try-with-resources guarantees closure order (reverse of opening) and correctly handles suppressed exceptions from `close()` failures.

### Retry with Backoff

Exponential backoff (1s → 2s → 4s) prevents thundering herd on recovering services. Always cap retry count and wrap final failure as a domain exception with all attempt details preserved.

**68%**

**Healthcare Software Failures**

Attributed to unhandled exceptions (HIMSS, 2023)

**1.5M**

**Patients Affected Annually**

By pharmacy dispensing errors (Institute of Medicine)

**40%**

**Faster Resolution**

With well-designed exception hierarchies (Gartner, 2022)

**7,000**

**Annual Deaths**

Linked to medication errors, many software-preventable (FDA)